

# COMPTE-RENDU TECHNIQUE

CodeFlow — Plateforme d'apprentissage du développement assistée par IA

## Présentation du projet

Dans le cadre de ma première année de Bachelor Développeur Full Stack, j'ai réalisé un projet individuel nommé **CodeFlow**. L'objectif était de mettre en pratique les notions apprises durant l'année tout en découvrant de nouvelles technologies non abordées en cours.

Le projet devait me permettre de développer une application complète, allant de la conception de la base de données jusqu'au déploiement en production.

## Présentation de l'application

CodeFlow est une plateforme web d'apprentissage du développement informatique assistée par intelligence artificielle. L'objectif principal de l'application est de permettre à tout type de public d'apprendre à programmer ou de maintenir ses compétences à jour grâce à des exercices générés dynamiquement.

L'application s'adresse particulièrement aux profils suivants :

- Aux débutants souhaitant apprendre à programmer ;
- Aux étudiants cherchant à s'entraîner ;
- Aux développeurs confirmés souhaitant approfondir certaines notions ;
- Aux développeurs expérimentés désirant rester à jour grâce à la veille technologique.

## Problématique

Aujourd'hui, de nombreuses ressources permettent d'apprendre à programmer. Cependant, elles présentent souvent certaines limites : les exercices sont généralement statiques, le niveau n'est pas adapté au profil de l'utilisateur, les corrections sont limitées et il n'existe pas toujours de suivi personnalisé.

***L'objectif de CodeFlow :** Proposer une solution capable de générer des exercices adaptés au niveau de chacun, fournir un retour personnalisé, proposer une progression évolutive et accompagner l'utilisateur tout au long de son apprentissage.*

## Choix techniques

### Front-end

Le projet a été développé avec une pile moderne et performante :

**Pourquoi Next.js ?** Le choix de Next.js s'est imposé pour plusieurs raisons. Tout d'abord, le framework offre une architecture full-stack permettant de développer le front-end et le back-end dans un même projet. Ensuite, Next.js apporte de nombreuses fonctionnalités nativement : système de routage automatique, gestion des métadonnées, optimisation SEO, rendu hybride et API intégrée.

Le référencement naturel a également été un critère important. Contrairement à une application React classique, Next.js permet une meilleure indexation par les moteurs de recherche.

**TypeScript** : L'application a été développée en TypeScript afin de bénéficier d'un typage fort, d'une meilleure maintenabilité, d'une détection précoce des erreurs et d'un meilleur confort de développement.

**Interface utilisateur** : L'interface a été développée avec Tailwind CSS. Cette bibliothèque a été choisie pour sa rapidité de développement, sa simplicité, son excellente intégration avec React et Next.js, et sa capacité à créer rapidement des interfaces responsives adaptées aux ordinateurs, tablettes et smartphones.

## Architecture de l'application

L'architecture de CodeFlow repose sur plusieurs couches distinctes :

- **Client** : L'utilisateur interagit avec l'application à travers son navigateur web.
- **Serveur Next.js** : Le serveur traite l'authentification, la génération des exercices, la communication avec l'intelligence artificielle et les accès à la base de données.
- **Base de données** : Les données sont stockées dans une base PostgreSQL hébergée sur Supabase.
- **Intelligence artificielle** : L'IA est utilisée pour générer les exercices, corriger le code, proposer des conseils et déterminer la progression de l'utilisateur.

Utilisateur → Next.js → API IA → Base de données → Affichage du résultat

## Technologies utilisées & Infrastructure

**Base de données** : PostgreSQL géré via la plateforme **Supabase**. L'accès aux données s'effectue à travers l'ORM **Prisma v6**, choisi afin d'éviter l'écriture de requêtes SQL complexes et de bénéficier d'un typage automatique, d'une meilleure lisibilité, d'un système de migrations et d'une maintenance simplifiée.

**Déploiement** : L'application est déployée de manière continue sur **Vercel**. Le code source est hébergé sur GitHub et utilise un pipeline CI/CD automatique via GitHub Workflows permettant un redéploiement transparent à chaque mise à jour.

## Authentification et sécurité

L'application possède deux méthodes d'authentification gérées par **Auth.js** (anciennement NextAuth) à l'aide de JSON Web Tokens (JWT) avec une validité de session fixée à 24 heures :

- Authentification classique (Email / Mot de passe) ;
- Authentification tierce via GitHub grâce au protocole OAuth.

Les mots de passe sont validés côté client, hachés avec **bcrypt** (facteur de 10 salt rounds) avant leur enregistrement, et ne sont jamais stockés en clair. Des expressions régulières (Regex) robustes vérifient la conformité des données d'inscription.

Certaines routes critiques (*dashboard*, *compte*, *veille*, *exercices*) sont protégées par un middleware garantissant qu'un utilisateur ne peut accéder qu'à ses propres données. De plus, les variables sensibles sont rigoureusement isolées dans un fichier d'environnement.

## Fonctionnement de l'intelligence artificielle

L'application exploite le modèle de pointe **Llama-3.3-70B-Versatile** via l'API ultra-rapide de **Groq**. L'IA a été baptisée **Lumina**. Son rôle englobe la génération d'exercices, l'analyse du code, l'attribution d'une note pédagogique et la formulation de conseils.

Le système prend en compte l'historique récent de l'utilisateur (les 20 derniers exercices) afin de proposer un système d'apprentissage adaptatif performant basé sur le contexte.

### Génération des exercices

L'utilisateur sélectionne un langage parmi les 10 disponibles (*JavaScript*, *TypeScript*, *Python*, *Java*, *C#*, *Go*, *Rust*, *C++*, *PHP*, *Swift*) et un niveau de difficulté (*Easy*, *Medium*, *Hard*, *Expert*). Lumina reçoit ces paramètres associés à des règles pédagogiques strictes et renvoie un objet structuré comprenant le titre, l'énoncé, les résultats attendus et les notions travaillées.

## Évaluation du code et Suivi

**Évaluation** : L'utilisateur rédige son code directement dans l'application grâce à **Monaco Editor** (moteur de VS Code). Lors de la soumission via le bouton « Forger », le code est analysé logiquement par Lumina sans exécution directe. Un score est attribué, et l'exercice est validé si la note atteint ou dépasse **80 %**.

**Suivi de progression** : Le tableau de bord affiche dynamiquement des statistiques calculées à la volée (langages les plus utilisés, historique, exercices en cours) permettant un suivi précis de l'évolution de l'apprenant.

## Veille technologique

Une section dédiée à la veille technologique est connectée à l'API **Dev.to**. L'utilisateur peut y consulter des articles techniques, les filtrer par langage et, de façon innovante, demander à l'IA de générer un exercice pratique directement basé sur le sujet de l'article sélectionné.

## Difficultés, Perspectives et Conclusion

**Difficultés rencontrées** : Le développement a nécessité un travail important de *Prompt Engineering* pour calibrer les réponses de l'IA. Des limitations initiales avec l'API Gemini ont mené à une migration réussie vers Groq. La configuration de l'éditeur Monaco a également représenté un défi technique notable.

**Perspectives d'amélioration** : Les évolutions futures prévoient la mise en place d'un système de *rate limiting*, l'ajout de fonctionnalités communautaires (badges, classements), et l'intégration de nouveaux langages de programmation.

**Conclusion** : CodeFlow constitue une expérience full-stack particulièrement formatrice, de la modélisation à la mise en production. Ce projet de Bachelor valide des compétences approfondies en architecture logicielle, sécurité, gestion des bases de données et intégration de modèles d'IA modernes.